

数値解析用プログラミング言語と連動したアニメーション図説フレームワークの開発に関する研究

西村美春

情報デザイン・コミュニケーション工学コース 数理波動システム研究室

1 背景・目的

現在、教育資料や発表資料などのプレゼンテーション資料は、動画編集ソフトやプレゼンテーション作成ソフトを用いて作成されている。代表的なプレゼンテーション作成ツールとして、PowerPoint[1] が広く利用されており、静的な資料を作成する際には操作が容易で、視覚的にわかりやすい資料を作成できる点で優れている。一方で、図形を視覚的操作によって作成するため、多数の図形を含む図や、寸法・角度・サイズなどの比率を正確に設定することが困難である。また、数式や数値解析で用いたコードから直接グラフを生成することはできず、計算結果を一度数値データとして出力し、Excel 等の表計算ソフトを介して可視化する必要がある。さらに、同一または類似したアニメーションであっても、再利用が難しいという問題がある。

既存のアニメーション作成ツールとして Manim[2] が挙げられる。Manim は Python コードを用いて数式やグラフを記述し、それらをアニメーションとして動画化できるアニメーション作成ライブラリである。数式表現に強く、教育用途や研究発表において有用である。しかし使用には Python による記述が必要であり、数値解析用に作成した C 言語や Fortran のコードをそのまま活用することはできない。

これらの問題を解決するため、本研究では新たなアニメーション図説フレームワークを構築することを目的とする。本フレームワークは、数値解析用のプログラミング言語 (Aqualis) [3] と共通の記述によって、スライド資料を作成できる仕組みを実現する。

2 使用言語

本フレームワークは F# のクラスライブラリとして開発した。F# は関数型プログラミング言語であり、コードが簡潔で可読性の高く、複雑な処理の記述に適している。また、F# はコンパイラとインタプリタの両面を持っており、本フレームワークはコンパイルされたライブラリ (.fsx ファイル) として実装し、アニメーションコンテンツの

記述は F# スクリプト (.fsx ファイル) により行っている。これにより、ユーザは必要最低限のコードのみを記述すればよく、記述量の削減と可読性の向上が可能となる。

3 フレームワークの実装

図 1 は、本研究で提案するアニメーション図説フレームワークにおけるアニメーション資料生成の流れを示したものである。以下では、この流れについて説明する。

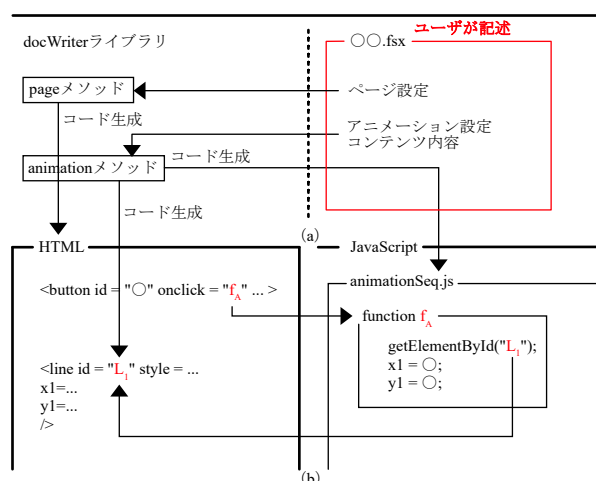


図 1 ソースファイル間の関係図 (a) コード生成まで (b) アニメーション描画時の処理

ソースコード 1 アニメーション描画領域の生成

```
html.animation outputdir "line" {sX=700;
sY=440; mX=160; mY=590;
backgroundColor="#bbeeff"} p
(100,590) <| fun (f,p) ->
```

fsx ファイルにおいて、ソースコード 1 のような記述を行うことで、fs ファイルに定義されたアニメーション描画領域を生成する処理が呼び出され、描画領域が生成される。html.animation は、HTML 形式のアニメーション出力を生成するための関数である。ユーザは生成されたファイルの出力先ディレクトリ、出力されるアニメー

ションの識別名、スライド領域の大きさ、表示領域における基準座標、背景色、ならびに start ボタンおよび reset ボタンの表示位置を指定することができる。また、図形の描画には SVG タグを用いており、図形の長さやサイズ、位置などの各種パラメータは、ユーザが数値で指定する。これらの情報をもとに、フレームワーク内部の処理が実行され、HTML ファイルおよび JavaScript ファイルが自動的に生成される。

生成されたアニメーションでは、図形、テキスト、数式をコンテンツとして扱うことができる。また、コンテンツのアニメーション処理は数式によって動きを定義することができ、その記述をもとに JavaScript ファイルが生成される。さらに、本フレームワークではアニメーション制御において、複数のコンテンツを同時に動作させることも、順番に動作させていくことも可能である。アニメーションの実行時には、JavaScript の `setTimeout` により一定時間間隔で描画処理が繰り返し呼び出され、各時刻における座標や寸法が再計算される。JavaScript ファイル内で計算された座標位置が HTML 側に反映されることで、アニメーションが制御されている。

4 結果

前述したアニメーション資料生成の流れを、ソースコード 2 に示す直線のスタイルおよび動きを用いて実行すると、直線が上下に揺れるアニメーションが表示される。

ソースコード 2 直線のアニメーション

```
1 let line0 = f.animationLine Style[stroke.
  width 3.0; stroke.color "#ff4c4c"]
2 f.seq {FrameTime=30; FrameNumber=1000} <|
  fun _ ->
3   line0.P {Start = {X = fun _ -> Dbl_e 200
4     Y = fun t -> 240+100*
      asm.sin(-2.0*Math.PI*t/100)}}
5   End = {X = fun _ -> Dbl_e 400
6     Y = fun t -> 240+100*asm.
      sin(2.0*Math.PI*t/100)}}}
```

ここでは、正弦関数を用いて直線の始点および終点の Y 座標を時間に応じて変化させ、滑らかな上下運動を実現している。ソースコード 2 の始点は

$$x(t) = 200, y(t) = 240 + 100 \sin\left(-\frac{2.0\pi t}{100}\right) \quad (1)$$

となっている。

また、図 2 の上部に示すアニメーションは繰り返し構文を用いて同一の図形を複数生成し、それらを同時に動作させた例である。このように、本フレームワークでは簡潔な記述によって、複数の図形を用いた動的なアニメーションを容易に作成することができる。さらに、図 2 の下部に示すプレゼンテーション資料全体では、ページ遷移用のボタンに加え、キャラクター、音声、字幕などの表示も可能で、オンデマンド教材としても使用できる。

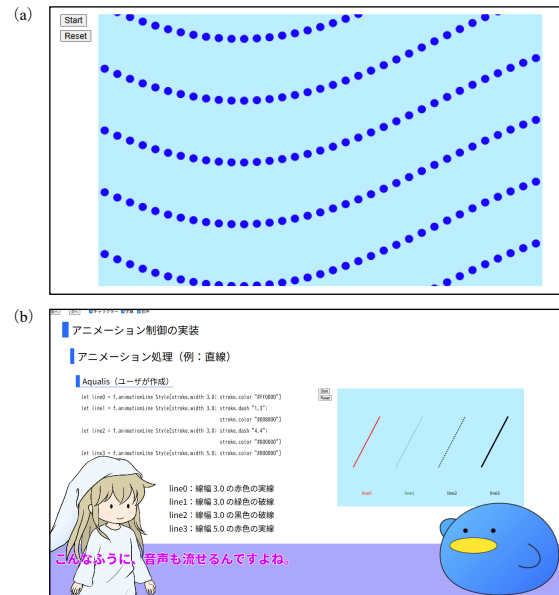


図 2 (a) 繰り返し構文を用いたアニメーション (b) プレゼンテーション資料全体図

5 まとめ

本研究では、アニメーション図説フレームワークを開発した。本フレームワークにより、数値解析用プログラミング言語 (Aqualis) と共通のコードを用いて、図形や数式を含むアニメーションスライドを作成することが可能となった。これにより、解析結果の視覚的な説明を効率的に行う手法を示した。

参考文献

- [1] Microsoft, “Microsoft powerpoint.”
<https://www.microsoft.com/ja-jp/microsoft-365/powerpoint>.
- [2] Manim Community, “Manim community.”
<https://www.manim.community/>.
- [3] 杉坂 純一郎, “Sugisaka/Aqualis.”
<https://github.com/Sugisaka/Aqualis>.